Supplementary Material for *SART: Sign-Absolute Reformulation Theory for Binary Variable Reduction in Neural Network Verification*

JIN XU, Tongji University, China
MIAOMIAO ZHANG*, Tongji University, China
BOWEN DU*, Tongji University, China

A A working example

Given the complete activation vector $\mathbf{a}^{(\ell-1)}$ from layer $\ell-1$, its positive and negative components are defined via a masking operation as follows:

$$\begin{aligned} \mathbf{a}_{+}^{(\ell-1,j)} &= \left(\mathbf{a}^{(\ell-1)} \cdot \mathbf{1}^{\top} \odot \left[\mathrm{Mask}(\mathbf{W}_{+}^{(\ell)})\right]^{\top}\right)[:, j], \\ \mathbf{a}_{-}^{(\ell-1,j)} &= \left(\mathbf{a}^{(\ell-1)} \cdot \mathbf{1}^{\top} \odot \left[\mathrm{Mask}(\mathbf{W}_{-}^{(\ell)})\right]^{\top}\right)[:, j]. \end{aligned} \tag{1}$$

Symbol Explanations:

- $\mathbf{a}^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1}}$: Input activation vector.
- $\mathbf{1} \in \mathbb{R}^{n_\ell}$: All-ones row vector for broadcasting the input activation dimension.
- $\odot$: Operator for element-wise multiplication.
- $\mathrm{Mask}(\mathbf{W}_{+}^{(\ell)}) \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$: Positive weight mask matrix indicating non-negative elements in the weight matrix. Similarly for negative weights.
- $[:, j]$: Operator for extracting the j-th column of a matrix.
- $\mathbf{a}_{+}^{(\ell-1,j)} \in \mathbb{R}^{n_{\ell-1}}$: Sparse input activation vector corresponding to the j-th output neuron, reflecting activation values after positive weight screening. Similarly for negative weights.

We present a working example in Figure 2 in the main paper, which demonstrates the case when $\ell = 2$, the computational framework begins with defining the input activation vector $\mathbf{a}^{(\ell-1)} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$ and the weight matrix $\mathbf{W}^{(\ell)} = \begin{bmatrix} 0.1 & -0.3 & 0.6 \\ 0.2 & 0.4 & -0.7 \\ 0.3 & -0.5 & -0.8 \end{bmatrix}$. The weight matrix is decomposed into positive and negative components based on sign polarity:

$$\mathbf{W}_{+}^{(\ell)} = \begin{bmatrix} 0.1 & 0 & 0.6 \\ 0.2 & 0.4 & 0 \\ 0.3 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_{-}^{(\ell)} = \begin{bmatrix} 0 & -0.3 & 0 \\ 0 & 0 & -0.7 \\ 0 & -0.5 & -0.8 \end{bmatrix},$$

*Corresponding authors.

Authors' Contact Information: Jin Xu, jin.xu@tongji.edu.cn, Tongji University, Shanghai, China; Miaomiao Zhang, Tongji University, Shanghai, China, miaomiao@tongji.edu.cn; Bowen Du, Tongji University, Shanghai, China, bowendu@tongji.edu.cn.

where the corresponding masks $\text{Mask}(\mathbf{W}_+^{(\ell)}) = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$ and $\text{Mask}(\mathbf{W}_-^{(\ell)}) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ isolate weight polarities. The input activation is expanded via outer product $\mathbf{a}^{(\ell-1)} \cdot \mathbf{1}^\top = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 0 & 0 & 0 \end{bmatrix}$, which is then element-wise multiplied ($\odot$) by the transposed masks to generate masked activation matrices:

$$\mathbf{A}_+^{(\ell-1)} = \left(\mathbf{a}^{(\ell-1)} \cdot \mathbf{1}^\top\right) \odot \left[\text{Mask}\left(\mathbf{W}_+^{(\ell)}\right)\right]^\top = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

$$\mathbf{A}_-^{(\ell-1)} = \left(\mathbf{a}^{(\ell-1)} \cdot \mathbf{1}^\top\right) \odot \left[\text{Mask}\left(\mathbf{W}_-^{(\ell)}\right)\right]^\top = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}$$

Column-wise extraction from these matrices yields component activation vectors. For $j = 1$, the positive and negative components are $\mathbf{a}_+^{(\ell-1,1)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{a}_-^{(\ell-1,1)} = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}$; for $j = 2$, $\mathbf{a}_+^{(\ell-1,2)} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$ and $\mathbf{a}_-^{(\ell-1,2)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$; and for $j = 3$, $\mathbf{a}_+^{(\ell-1,3)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{a}_-^{(\ell-1,3)} = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}$. Pre-activations are computed via $z_{+j}^{(\ell)} = \mathbf{W}_{+,j}^{(\ell)} \cdot \mathbf{a}_+^{(\ell-1,j)}$ and $z_{-j}^{(\ell)} = \mathbf{W}_{-,j}^{(\ell)} \cdot \mathbf{a}_-^{(\ell-1,j)}$. For neuron $j = 1$, this yields:

$$z_{+1}^{(\ell)} = 0.1 \times 1 = 0.1, \quad z_{-1}^{(\ell)} = -0.3 \times 2 = -0.6;$$

for $j = 2$:

$$z_{+2}^{(\ell)} = 0.2 \times 1 + 0.4 \times 2 = 1.0, \quad z_{-2}^{(\ell)} = 0;$$

and for $j = 3$:

$$z_{+3}^{(\ell)} = 0.3 \times 1 = 0.3, \quad z_{-3}^{(\ell)} = -0.5 \times 2 = -1.0.$$

Aggregating these results produces the total pre-activation vector:

$$\mathbf{z}^{(\ell)} = \begin{bmatrix} 0.1 - 0.6 \\ 1.0 + 0 \\ 0.3 - 1.0 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 1.0 \\ -0.7 \end{bmatrix}.$$

Direct verification via matrix multiplication $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)}$ confirms consistency:

$$\mathbf{z}^{(\ell)} = \begin{bmatrix} 0.1 \cdot 1 + (-0.3) \cdot 2 + 0.6 \cdot 0 \\ 0.2 \cdot 1 + 0.4 \cdot 2 + (-0.7) \cdot 0 \\ 0.3 \cdot 1 + (-0.5) \cdot 2 + (-0.8) \cdot 0 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 1.0 \\ -0.7 \end{bmatrix}.$$

The equivalence $\mathbf{z}^{(\ell)} = \mathbf{z}_+^{(\ell)} + \mathbf{z}_-^{(\ell)}$ validates the decomposition methodology, demonstrating precise alignment between component-wise computations and direct matrix operations.

B Proofs

B.1 Proof of Theorem 4.1 in the main paper

Proof. We proceed by contradiction. Suppose there exists an index j_0 such that $u_{j_0}^* > 0$ and $v_{j_0}^* > 0$, which violates the complementarity condition. We construct a new solution depending on the relative magnitude of $u_{j_0}^*$ and $v_{j_0}^*$.

Case 1: If $u^*_{j_0} \le v^*_{j_0}$, define the new solution (X, U, V) as

$$u_k = \begin{cases} 0, & k = j_0 \\ u^*_k, & k \ne j_0 \end{cases}, \quad v_k = \begin{cases} v^*_{j_0} - u^*_{j_0}, & k = j_0 \\ v^*_k, & k \ne j_0 \end{cases}.$$

This solution satisfies non-negativity: when $k = j_0$, $v^*_{j_0} - u^*_{j_0} \ge 0$ (guaranteed by the assumption $u^*_{j_0} \le v^*_{j_0}$); when $k \ne j_0$, non-negativity is inherited from the original solution. Furthermore, the equality constraint $\sum_{i=1}^{n} d_{ki} x^*_i + u_k - v_k = 0$ holds:

- For $k = j_0$, we have $\sum_{i=1}^{n} d_{j_0 i} x^*_i + 0 - (v^*_{j_0} - u^*_{j_0}) = \sum_{i=1}^{n} d_{j_0 i} x^*_i + u^*_{j_0} - v^*_{j_0} = 0$;
- For $k \ne j_0$, the constraint remains unchanged.

The new objective value is

$$f(X, U, V) - f(X^*, U^*, V^*) = \left(v^*_{j_0} - u^*_{j_0}\right) - \left(u^*_{j_0} + v^*_{j_0}\right) = -2u^*_{j_0} < 0,$$

which is strictly smaller than the objective value of the original solution, contradicting the optimality of (X^*, U^*, V^*).

Case 2: If $u^*_{j_0} > v^*_{j_0}$, define the new solution

$$v_k = \begin{cases} 0, & k = j_0 \\ v^*_k, & k \ne j_0 \end{cases}, \quad u_k = \begin{cases} u^*_{j_0} - v^*_{j_0}, & k = j_0 \\ u^*_k, & k \ne j_0 \end{cases}.$$

Non-negativity holds (since $u^*_{j_0} - v^*_{j_0} > 0$), and the constraints are verified similarly to Case 1. The change in the objective value is

$$f(X, U, V) - f(X^*, U^*, V^*) = \left(u^*_{j_0} - v^*_{j_0}\right) - \left(u^*_{j_0} + v^*_{j_0}\right) = -2v^*_{j_0} < 0,$$

which also contradicts the optimality. Therefore, the assumption is false, and for any k, it must hold that $u^*_k \cdot v^*_k = 0$. In this case, (X^*, U^*, V^*) satisfies all constraints of T_a, and its objective value is the same as in Q_a, thus it is also the optimal solution to T_a. □

B.2 Proof of Theorem 4.2 in the main paper

Proof. We proceed by contradiction. Suppose that (X', U', V') is an optimal solution to Problem T_a, but X' is not an optimal solution to Problem P_a. Then there exists a feasible solution $\hat{X}$ to Problem P_a such that the objective value of $\hat{X}$ is strictly smaller than that of X', i.e.,

$$f(\hat{X}) = \sum_{i=1}^{n} c_i \hat{x}_i + \sum_{k=1}^{K} \left| \sum_{i=1}^{n} d_{ki} \hat{x}_i \right| < \sum_{i=1}^{n} c_i x'_i + \sum_{k=1}^{K} \left| \sum_{i=1}^{n} d_{ki} x'_i \right| = f(X').$$

Based on the feasible solution $\hat{X}$, we construct auxiliary variables $\hat{U}$ and $\hat{V}$ for Problem T_a. For each constraint $k = 1, \ldots, K$, define them according to the sign of the linear combination $\sum_{i=1}^{n} d_{ki} \hat{x}_i$ as follows:

- If $\sum_{i=1}^{n} d_{ki} \hat{x}_i \ge 0$, define $\hat{u}_k = 0$, $\hat{v}_k = \sum_{i=1}^{n} d_{ki} \hat{x}_i$;
- If $\sum_{i=1}^{n} d_{ki} \hat{x}_i < 0$, define $\hat{u}_k = -\sum_{i=1}^{n} d_{ki} \hat{x}_i$, $\hat{v}_k = 0$.

We first verify that $(\hat{X}, \hat{U}, \hat{V})$ satisfies all constraints of Problem T_a:

(1) Linear equality constraints: In both cases, we have $\sum_{i=1}^{n} d_{ki}\hat{x}_i + \hat{u}_k - \hat{v}_k = 0$.

(2) Non-negativity constraints: By definition, $\hat{u}_k \geq 0$, $\hat{v}_k \geq 0$, and $\hat{X} \geq 0$.

(3) Complementarity slackness constraints: In both cases, we have $\hat{u}_k \cdot \hat{v}_k = 0$.

Therefore, $(\hat{X}, \hat{U}, \hat{V})$ is a feasible solution to Problem T_a.

Next, consider the objective value of this feasible solution. By the definitions of $\hat{u}_k$ and $\hat{v}_k$, we have $\hat{u}_k + \hat{v}_k = \left|\sum_{i=1}^{n} d_{ki}\hat{x}_i\right|$. Thus, the objective value of Problem T_a becomes:

$$f(\hat{X}, \hat{U}, \hat{V}) = \sum_{i=1}^{n} c_i\hat{x}_i + \sum_{k=1}^{K}(\hat{u}_k + \hat{v}_k) = \sum_{i=1}^{n} c_i\hat{x}_i + \sum_{k=1}^{K}\left|\sum_{i=1}^{n} d_{ki}\hat{x}_i\right| = f(\hat{X}).$$

From the initial assumption $f(\hat{X}) < f(X')$, and given that (X', U', V') is an optimal solution to Problem T_a, it follows that:

$$f(\hat{X}, \hat{U}, \hat{V}) = f(\hat{X}) < f(X') = f(X', U', V').$$

This inequality implies that the objective value of $(\hat{X}, \hat{U}, \hat{V})$ is strictly smaller than that of the optimal solution (X', U', V'), contradicting the optimality of (X', U', V'). Hence, the assumption is false, and X' must be an optimal solution to Problem P_a. □

B.3 Proof of Theorem 4.4 in the main paper

Proof. We construct a counterexample. Let the dimension parameter $n = 1$ and the number of absolute value terms $K = 1$. Then, the original problem (problem P_s) is given by:

$$\min_{x_1} \{0 \cdot x_1 - |x_1|\} \quad \text{s.t.} \quad x_1 = 0,\ x_1 \geq 0.$$

The feasible region of this problem contains only one solution $x_1 = 0$, with the objective value $0 - |0| = 0$.

Now consider its relaxed problem (problem Q_s):

$$\min_{x_1, u_1, v_1} \{0 \cdot x_1 - (u_1 + v_1)\} \quad \text{s.t.} \quad x_1 = 0,\ x_1 + u_1 - v_1 = 0,\ x_1, u_1, v_1 \geq 0.$$

From the constraints, we obtain:

$$x_1 = 0 \quad \text{and} \quad 0 + u_1 - v_1 = 0 \implies u_1 = v_1.$$

Substituting into the objective function yields:

$$f(x_1, u_1, v_1) = 0 \cdot 0 - (u_1 + v_1) = -(u_1 + v_1) = -2u_1.$$

Since u_1 can increase arbitrarily without a lower bound other than zero, the objective value tends to $-\infty$ as $u_1 \to +\infty$, indicating that problem Q_s does not have a bounded optimal solution.

Now consider adding the complementarity condition (i.e., problem T_s):

$$u_1 \cdot v_1 = 0, \quad u_1 \geq 0, \quad v_1 \geq 0.$$

Combined with the previously derived condition $u_1 = v_1$, the complementarity constraint requires $u_1 \cdot u_1 = u_1^2 = 0$, thus the only feasible solution is $u_1 = v_1 = 0$, corresponding to the objective value $f(0, 0, 0) = -2 \cdot 0 = 0$. However, the solutions in problem Q_s that drive the objective towards $-\infty$ require $u_1 = v_1 \to +\infty$, which clearly violates the complementarity condition $u_1 \cdot v_1 = 0$.

In conclusion, the unbounded "optimal solution" of problem Q_s is not within the feasible region of problem T_s, verifying the theorem. □

B.4 Proof of Theorem 4.5 in the main paper

Proof. (By contradiction, paralleling the additive version)

Suppose X^* is not the optimal solution to problem P_s, then there exists a feasible solution $\hat{X}$ such that

$$f(\hat{X}) \;=\; \sum_{i=1}^{n} c_i \hat{x}_i \;-\; \sum_{k=1}^{K} \left| \sum_{i=1}^{n} d_{ki} \hat{x}_i \right| \;<\; f(X^*).$$

We construct $\hat{U}, \hat{V}$ in the same manner as in the additive version (with identical definitions of signs):

- If $\sum_{i=1}^{n} d_{ki}\, \hat{x}_i \geq 0$: set $\hat{u}_k = 0,\ \hat{v}_k = \sum_{i=1}^{n} d_{ki}\, \hat{x}_i$.
- If $\sum_{i=1}^{n} d_{ki}\, \hat{x}_i < 0$: set $\hat{u}_k = -\sum_{i=1}^{n} d_{ki}\, \hat{x}_i,\ \hat{v}_k = 0$.

This construction also satisfies all constraints of T_s (the verification process is identical to the additive version). The objective function relation becomes

$$\begin{aligned} f(\hat{X}, \hat{U}, \hat{V}) &= \sum_{i=1}^{n} c_i \hat{x}_i \;-\; \sum_{k=1}^{K} \left(\hat{u}_k + \hat{v}_k\right) \\ &= \sum_{i=1}^{n} c_i \hat{x}_i \;-\; \sum_{k=1}^{K} \left| \sum_{i=1}^{n} d_{ki} \hat{x}_i \right| = f(\hat{X}). \end{aligned}$$

By the assumption $f(\hat{X}) < f(X^*)$, it follows that

$$f(\hat{X}, \hat{U}, \hat{V}) \;<\; f(X^*, U^*, V^*).$$

This contradicts the assumption that (X^*, U^*, V^*) is the optimal solution to T_s. Therefore, X^* must be the optimal solution to P_s. □

B.5 Proof of Theorem 4.7 in the main paper

Proof. We prove by contradiction. Suppose the constraint can be expressed purely by linear programming, then its feasible region must be convex. Select an arbitrary index l_0 corresponding to one absolute value term, and construct two feasible solutions:

- $X_1^{(l_0)}$: satisfying $\sum_i h_{l_0 i}^{(t)} x_i = a > 0$, and for all $l \neq l_0$, $\sum_i h_{li}^{(t)} x_i = 0$;
- $X_2^{(l_0)}$: satisfying $\sum_i h_{l_0 i}^{(t)} x_i = -a < 0$, and the other terms are the same as above.

Remark. The two solutions $X_1^{(l_0)}$ and $X_2^{(l_0)}$ are not assumed to exist a priori; they are deliberately **constructed** to satisfy the constraints. Because X is chosen in this way by construction, such a pair can always be produced; no appeal to "natural" existence is required.

Notation: x_{1i} and x_{2i} denote the i-th components of $X_1^{(l_0)}$ and $X_2^{(l_0)}$, respectively; likewise, $x_{\theta i}$ denotes the i-th component of X_θ.

Both solutions satisfy the constraint and

$$\sum_i g_i^{(t)} x_{1i} = \sum_i g_i^{(t)} x_{2i} = q^{(t)} - a.$$

Consider their convex combination:

$$X_\theta = \theta X_1^{(l_0)} + (1-\theta) X_2^{(l_0)}, \quad \theta \in (0,1).$$

Then,

$$\sum_i g_i^{(t)} x_{\theta i} = q^{(t)} - a,$$
$$\sum_i h_{l_0 i}^{(t)} x_{\theta i} = a(2\theta - 1),$$
$$\sum_i h_{li}^{(t)} x_{\theta i} = 0, \quad \forall l \neq l_0.$$

Substituting into the constraint yields

$$\sum_i g_i^{(t)} x_{\theta i} + \sum_{l=1}^{L_t} \left| \sum_i h_{li}^{(t)} x_{\theta i} \right| = (q^{(t)} - a) + |a(2\theta - 1)| < q^{(t)},$$

since $|2\theta - 1| < 1$ for all $\theta \in (0, 1)$.

Therefore, the convex combination point X_θ does not satisfy the constraint, implying the feasible region is nonconvex, contradicting the assumption. In conclusion, the feasible region defined by this constraint is inherently nonconvex. Moreover, the sign changes of each absolute value term independently affect convexity and cannot be represented using shared integer variables. To ensure an equivalent and complete model, it is necessary to introduce an individual binary variable $y_l^{(t)} \in \{0, 1\}$ for each absolute value term, accurately capturing its sign state. □

B.6 Proof of Theorem 4.15 in the main paper

To rigorously prove this minimality, we analyze the modeling complexity of equivalent problems P_s, P_a^{full}, and P_s^{full}. Since these share the same absolute-value structure, proving it for P_s suffices; the others follow similarly.

Proof (by contradiction). Suppose P_s contains K absolute-value terms. A complete encoding must use binary variables to represent the sign of each term. Under the assumption $P \neq NP$, we show that the number of binary variables m satisfies $m \geq K$. Assume, for contradiction, that a complete model with $m < K$ binary variables can be solved in polynomial time. Since each binary variable encodes two states, such a model can represent at most $2^m < 2^K$ sign configurations, insufficient to cover all sign assignments. Consider a polynomial-time reduction from an NP-complete problem (e.g., Partition) to P_s, where each partition corresponds to a sign configuration in P_s. If the model cannot represent all configurations, it cannot correctly solve Partition. Hence, if P_s were solvable in polynomial time with fewer than K binary variables, Partition would also be polynomial-time solvable, implying $P = NP$, a contradiction. Therefore, any complete model for P_s requires at least K binary variables. The proof is complete. □

Theorem 4.15 states a structural minimality result. For any problem of the form P_s with K pairwise independent absolute-value atoms $\{|d_k^\top X|\}_{k=1}^K$ (each encoding an independent sign choice), any reformulation that is complete and correct *at the instance level* must distinguish all 2^K sign patterns. Consequently, at least K binary variables-equivalently, K bits of information-are necessary. If fewer than K binaries are used, some patterns that should be distinguishable are inevitably folded into the same code, which breaks completeness or correctness on certain instances. This lower bound is *structural.* This statement does not rely on complexity-theoretic claims (e.g., "MILP with absolute values is NP-complete"). As an independent auxiliary check, the appendix provides a Partition $\rightarrow P_s$ reduction B.7 that *exactly matches* the P_s template used here: set $c = 0$, take

$X = (r_1, t_1, \ldots, r_K, t_K) \in \mathbb{R}_+^{2K}$, impose

$$r_k + t_k = 1 \ (k = 1, \ldots, K), \qquad \sum_{k=1}^{K} a_k \, (r_k - t_k) = 0,$$

and use the objective $\min -\sum_{k=1}^{K} |d_k^\top X|$ with $d_k^\top X = r_k - t_k$. For this construction, deciding whether the optimal value equals $-K$ is equivalent to deciding the original Partition instance. Hence, if there existed a *uniform* complete-and-correct reformulation using $m < K$ binaries for all such P_s instances, one could decide Partition, contradicting the standard belief $P \neq NP$.

B.7 Partition $\to P_s$ Reduction

Step 1: Understanding PARTITION (instance and condition).
Input. Positive integers $a_1, \ldots, a_K \in \mathbb{Z}_{>0}$.
Decision. Does there exist a subset $S \subseteq \{1, \ldots, K\}$ such that

$$\sum_{k \in S} a_k \ = \ \sum_{k \notin S} a_k \, ?$$

Equivalent sign-vector form. Equivalently, is there $(s_k)_{k=1}^{K} \in \{\pm 1\}^K$ with

$$\sum_{k=1}^{K} a_k s_k \ = \ 0 \, ?$$

Step 2: Defining the target problem P_s (instance and condition).
Variables and definitions. For each k, introduce split variables $r_k, t_k \geq 0$ and set

$$z_k := r_k - t_k, \qquad X := (r_1, t_1, \ldots, r_K, t_K).$$

Linear equality constraints.

$$r_k + t_k = 1 \quad (k = 1, \ldots, K), \qquad \sum_{k=1}^{K} a_k \, z_k = 0.$$

Equivalently, these K+1 equations can be stacked as $AX = b$. Define rows d_k by $(d_k)_{r_k} = +1$, $(d_k)_{t_k} = -1$, others 0, so $d_k^\top X = z_k$. *Objective.*

$$\min f(X) \ = \ -\sum_{k=1}^{K} |z_k| \ = \ -\sum_{k=1}^{K} \left| d_k^\top X \right|.$$

Since $r_k, t_k \geq 0$ and $r_k + t_k = 1$, we have $|z_k| \leq 1$, hence

$$\min\Big(-\sum_k |z_k| \Big) \ \equiv \ \max\Big(\sum_k |z_k| \Big), \qquad \sum_k |z_k| \leq K.$$

Decision version. Decide whether the optimal value OPT equals $-K$.
Step 3: Designing the reduction (polynomial-time mapping).
Define

$$\mathcal{R} : \ \mathbb{Z}_{>0}^{K} \longrightarrow \mathcal{I}(P_s), \qquad (a_1, \ldots, a_K) \longmapsto \Big(X \in \mathbb{R}_+^{2K}, \ AX = b, \ \min -\sum_{k=1}^{K} |d_k^\top X| \Big),$$

where $X = (r_1, t_1, \ldots, r_K, t_K)$, the constraints are $r_k + t_k = 1$ for $k = 1, \ldots, K$ and $\sum_{k=1}^{K} a_k (r_k - t_k) = 0$, and each d_k has $+1$ at r_k and -1 at t_k. This construction is polynomial time: it produces $2K$ nonnegative variables, K+1 equality constraints, and the stated objective with coefficients in $\{-1, 0, 1\} \cup \{a_k\}$; the output size and bit-length are polynomial in the input.

Step 4: Proving equivalence (if and only if).

Let OPT denote the optimal value of the constructed instance.

(Sufficiency). If PARTITION is YES, pick $s \in \{\pm 1\}^K$ with $\sum_k a_k s_k = 0$. Set

$$r_k = \frac{1+s_k}{2}, \qquad t_k = \frac{1-s_k}{2} \;\Rightarrow\; z_k = r_k - t_k = s_k \in \{\pm 1\}.$$

This point is feasible and satisfies $\sum_k |z_k| = K$, thus OPT $= -K$.

(Necessity). If OPT $= -K$, then $\sum_k |z_k| = K$. As each $|z_k| \le 1$, we must have $|z_k| = 1$ for all k, i.e., $z_k \in \{\pm 1\}$. The coupling constraint gives $\sum_k a_k z_k = 0$. Setting $s_k := z_k$ yields a valid PARTITION sign vector; hence the instance is YES.

$$\text{OPT} = \begin{cases} -K, & \text{iff PARTITION is YES,} \\ > -K, & \text{iff PARTITION is NO.} \end{cases}$$

Step 5: Verifying polynomial time (reduction cost).

Size. $2K$ nonnegative variables and $K+1$ equality constraints; the objective is a sum of absolute values.

Bit length. Coefficients are in $\{-1, 0, 1\} \cup \{a_k\}$ with encoding length comparable to the input.

Time. Constructing variables, constraints, and the objective from (a_k) requires only linear-time assembly; hence $\mathcal{R}$ is a polynomial-time reduction.

Conclusion . Through $\mathcal{R}$, the PARTITION instance $(a_1, \ldots, a_K)$ and the P_s instance satisfy

$$\text{OPT} = -K \quad \Longleftrightarrow \quad \exists\, (s_k) \in \{\pm 1\}^K \text{ such that } \sum_{k=1}^{K} a_k s_k = 0.$$

Hence deciding PARTITION (YES/NO) is equivalent to deciding whether the optimal value of P_s equals $-K$.

B.8 Proof of Theorem 4.17 in the main paper

Proof. Define the objective function $f(a) = 2^a + 2^{S-a}$ where $a = N_+^{(m)}$. The first derivative:

$$f'(a) = 2^a \ln 2 - 2^{S-a} \ln 2, \tag{2}$$

yields critical point $a = S/2$. The second derivative:

$$f''(a) = 2^a (\ln 2)^2 + 2^{S-a} (\ln 2)^2 > 0, \tag{3}$$

confirms strict convexity. Thus:

- For even S, minimum occurs at $a = S/2$.
- For odd S, minima are at $a = \lfloor S/2 \rfloor$ and $a = \lceil S/2 \rceil$.

□

Multi-Layer Extension: For an m-layer network:

- Layers 1 to $m-1$: Complexity $O\left(2^{2(N_+^{(t)} + N_-^{(t)})}\right)$.
- Terminal layer: Optimized complexity $O\left(2^{N_+^{(m)}} + 2^{N_-^{(m)}}\right)$.

Total complexity becomes: $O\left(\prod_{t=1}^{m-1} 2^{2(N_+^{(t)} + N_-^{(t)})} \times \left(2^{N_+^{(m)}} + 2^{N_-^{(m)}}\right)\right)$.

Optimization Effects: For $S = 8$, symmetric allocation ($N_+^{(m)} = N_-^{(m)} = 4$) yields complexity $2^4 + 2^4 = 32$, while asymmetric allocation ($N_+^{(m)} = 5, N_-^{(m)} = 3$) gives $2^5 + 2^3 = 40$, achieving 20% reduction. The symmetry optimization applies recursively to all layers during verification, as each layer is treated as the "current terminal layer."

Summary. Enforcing sign symmetry ($\left|N_+^{(m)} - N_-^{(m)}\right| \leq 1$) minimizes verification complexity. This recursive optimization systematically reduces global complexity (e.g., 20% for $S = 8$), providing both mathematical guarantees and technical pathways for building efficiently verifiable deep learning models.

C Additional Experimental Results and Details

C.1 LayerABS Verification Results on VNNCOMP ERAN Models.

We conducted comparative experiments on the VNNCOMP ERAN models, and the results are shown in Table 1. MIPVerify results are not reported for these models for the following reason: MIPVerify is a prominent MILP method implemented in Julia that expects models in the `.mat` format. Converting the VNNCOMP `.pth` or `.onnx` files to `.mat` requires the name of the `Sequential` container used in the original training code. Since we only have access to the trained model files and cannot recover that name, the conversion cannot be completed. By contrast, the MNIST models in the main text were trained by us, so the container names are known and the models can be exported to `.mat`, enabling a direct comparison with MIPVerify. Table 1 summarizes VNNCOMP ERAN 5×100 ($\epsilon = 0.019$) and 8×100 ($\epsilon = 0.018$). LayerABS solves the most instances with fewer timeouts and competitive time, achieving *82 safe and 1 unsafe* (17 timeouts; mean 415.5 s) on 5×100, and *84 safe and 0 unsafe* (16 timeouts; mean 453.8 s) on 8×100, while Marabou and β CROWN BaB solve fewer cases and take longer. DeepPoly and α CROWN are fast but return many unknown outcomes.

Table 1. Verification Results on VNNCOMP ERAN Models

Method	Metric	VNNCOMP ERAN 5x100	VNNCOMP ERAN 8x100
	ϵ	0.019	0.018
LayerABS	safe | unsafe	**82 | 1**	**84 | 0**
	timeout	**17**	**16**
	Time(s)	**415.5**	**453.8**
MIPVerify	safe | unsafe	-	-
	timeout	-	-
	Time(s)	-	-
Marabou	safe | unsafe	59 | 4	55 | 2
	timeout	37	43
	Time(s)	758.0	882.4
β-CROWN BaB (CPU)	safe | unsafe	76 | 0	69 | 0
	timeout	24	31
	Time(s)	471.5	623.3
DeepPoly	safe	54	51
	unknown	46	49
	Time(s)	2.4	3.0
α-CROWN	safe	59	61
	unknown	41	39
	Time(s)	0.05	0.12

Note. MIPVerify results are omitted because the VNNCOMP ERAN models could not be converted to the .mat format required by MIPVerify.

C.2 PRIMA vs. Incomplete-LayerABS on VNNCOMP ERAN Models.

Results on VNNCOMP ERAN (incomplete methods, k=3). Incomplete–LayerABS is an incomplete verification method. On the VNNCOMP ERAN models we compare it with PRIMA, another

incomplete method; see Table 2. With broadly comparable counts of *safe* and *unknown* cases, Incomplete–LayerABS achieves shorter runtimes, reducing verification time by 11.2% to 23.6%, and therefore performs better under this setting.

Table 2. Comparison of Incomplete Methods on VNNCOMP ERAN Models (k=3): PRIMA vs Incomplete–LayerABS

Model	Metric	PRIMA (SOTA multi-neuron)	Incomplete-LayerABS
VNNCOMP ERAN 5x100 ϵ=0.019	safe ↑	88	**88**
	unknown ↓	12	**12**
	Time(s) ↓	197.5	**175.4** $\downarrow_{11.2\%}$
VNNCOMP ERAN 8x100 ϵ=0.018	safe ↑	86	85
	unknown ↓	14	15
	Time(s) ↓	365.5	**279.0** $\downarrow_{23.6\%}$

Table 3. Impact of hyperparameter k on VNNCOMP ERAN 5×100 and 8×100 using Incomplete-LayerABS

Model	k	1	2	3	4	5
VNNCOMP ERAN 5x100 ϵ=0.019	safe	83	83	88	88	90
	unknown	17	17	12	12	10
	Time(s)	76.7	118.7	175.4	199.1	205.5
VNNCOMP ERAN 8x100 ϵ=0.018	safe	81	83	85	87	87
	unknown	19	17	15	13	13
	Time(s)	186.3	247.1	279.0	342.4	358.6

Effect of the hyperparameter k. To assess the effect of the key hyperparameter k, we evaluate Incomplete-LayerABS on the VNNCOMP ERAN 5×100 and 8×100 models with $k \in \{1, 2, 3, 4, 5\}$, and measure its impact on verification accuracy (safe/unknown) and runtime. The results are reported in Table 3.

- As k increases from 1 to 5, *safe* increases and *unknown* decreases for both models, while runtime also increases.
- **VNNCOMP ERAN 5×100**: *safe* rises from 83 to 90 (+7) and runtime grows from 76.7 s to 205.5 s (about +168%). The main gain appears at k=3 (+5 safe), after which returns diminish. If a speed and accuracy tradeoff is preferred, k=3 is recommended; if higher accuracy is desired, k=5 adds only 6.4 s over k=4 and yields +2 additional safe.
- **VNNCOMP ERAN 8×100**: *safe* increases from 81 to 87 (+6) while runtime rises from 186.3 s to 358.6 s (about +93%). Accuracy plateaus at k=4; k=5 adds time without improving *safe*, making k=4 a preferable upper choice and k=3 the faster alternative.

C.3 LDSA on VNNCOMP ERAN: PRIMA Comparison and Sensitivity to k

Experimental setup. We conduct two experiments on the VNNCOMP ERAN models: a head-to-head comparison between Incomplete-LayerABS and PRIMA, and an evaluation of the effect of the key hyperparameter k on accuracy and runtime. Here, Incomplete-LayerABS is derived from Algorithm 1 by removing Stage 3. In the single-interaction setting, the abstraction method DeepPoly and Partial SART Encoding (denoted *pSART*) interact only once, and DeepPoly is not invoked inside the internal routine of pSART, which isolates the contribution of pSART. In a more practical implementation, since DeepPoly is inexpensive, we adopt a layerwise alternating procedure: at each

layer we run DeepPoly and then pSART and proceed with the updated intervals to the next layer. We refer to this procedure as *Layerwise DeepPoly SART Alternation* (*LDSA*); the abbreviations *LDSA* and *pSART* are used throughout the paper. The comparison with PRIMA is reported in Table 4, and the impact of k is summarized in Table 5.

Note. It is important to note that all results reported in the main text, for both the LayerABS and the Incomplete-LayerABS, use the single-interaction setting in which DeepPoly and pSART interact exactly once and no layerwise alternation is applied. This choice allows a fairer attribution of the improvements to the SART encoding itself. Under the LDSA (layerwise alternation) setting, the overall runtime would be even lower.

Table 4. Verification on VNNCOMP ERAN (LDSA setting, k=3): PRIMA vs Incomplete-LayerABS

Model	Metric	PRIMA (SOTA multi-neuron)	Incomplete-LayerABS
VNNCOMP ERAN 5x100 ϵ=0.019	safe ↑	88	85
	unknown ↓	12	15
	Time(s) ↓	197.5	**106.1** $_{\downarrow 46.3\%}$
VNNCOMP ERAN 8x100 ϵ=0.018	safe ↑	86	84
	unknown ↓	14	16
	Time(s) ↓	365.5	**207.0** $_{\downarrow 43.3\%}$

Table 5. Impact of Hyperparameter k on VNNCOMP ERAN 5×100 (LDSA setting, method: Incomplete-LayerABS)

Model	k	1	2	3	4	5
VNNCOMP ERAN 5x100 ϵ=0.019	safe	82	83	85	87	90
	unknown	18	17	15	13	10
	Time(s)	64.4	90.9	106.1	113.6	111.6

Overall analysis. In the VNNCOMP ERAN comparison (Table 4, k=3), Incomplete-LayerABS under the LDSA setting consistently shortens runtime while keeping accuracy broadly comparable to PRIMA: on 5×100 the runtime drops from 197.5 s to 106.1 s (46.3% reduction), and on 8×100 from 365.5 s to 207.0 s (43.3% reduction). The associated changes in *safe*/*unknown* are modest (from 88/12 to 85/15 on 5×100, and from 86/14 to 84/16 on 8×100), indicating a clear speed advantage with similar accuracy.

Table 5 reports the hyperparameter sensitivity. Increasing k from 1 to 5 raises *safe* from 82 to 90 and lowers *unknown* from 18 to 10; runtime grows and then stabilizes, peaking at 113.6 s for k=4 and slightly decreasing to 111.6 s for k=5. Overall, k=3 offers a favorable speed-accuracy balance, whereas k=5 achieves the highest *safe* with a comparable runtime.

Compared with the earlier single-interaction setting on 5×100 (76.7/118.7/175.4/199.1/205.5 s for k=1 . . . 5), the layerwise alternation in LDSA yields consistent speedups across all k values: 16.0%, 23.4%, 39.5%, 42.9%, and 45.7% reductions. Accuracy changes are small (−1, 0, −3, −1, 0 relative to the previous results). At k=5, both approaches reach *safe*= 90, while runtime is reduced from 205.5 s to 111.6 s, demonstrating that LDSA better leverages the strengths of abstraction and SART encoding.

C.4 Rationale for Choosing DeepPoly and Its Role in LayerABS

We select DeepPoly as the abstraction technique for three reasons: conceptual simplicity, ease of implementation with mature support from the open-source ERAN toolkit, and broad acceptance

in the verification community. To ensure fair and reproducible comparisons focused on encoding improvements, our complete baselines (Marabou and PRIMA) also use DeepPoly, which keeps all methods under the same abstraction setting. Although Table 4 in the main paper and Table 1 indicates that α-CROWN often outperforms DeepPoly, we did not adopt α-CROWN due to higher implementation complexity and fairness considerations.

Within LayerABS, abstraction serves two roles. First, it filters easy properties in Stage 1. Second, it provides initial intervals for Stage 2 to warm-start the exact encoding. Consequently, the abstraction is decoupled from our encoding technique, similar to the design in Marabou. In principle, replacing DeepPoly with a stronger abstraction would further improve overall performance. Since this work focuses on the encoding contribution, we adopt DeepPoly as a conservative and consistent baseline throughout the experiments.

C.5 The main configuration parameters of PRIMA

The main configuration parameters of PRIMA are provided in Table 6.

Table 6. PRIMA Parameter Setting

approx_k: True	delta: 0.3	epsilon: 0.015
numproc: 28	n_milp_refine: 1	timeout_lp: 30.0
timeout_final_lp: 30	timeout_final_milp: 30	s: -2
use_default_heuristic: True	sparse_n: 70	dataset: mnist
max_milp_neurons: 30	use_milp: True	timeout_milp: 30.0

C.6 Layer Level Statistics and Runtime Implications of SART Encoding

The Tables 7 and 8 report layer-wise internal statistics for verifying four properties (IDs 72, 18, 50, 89). We use the following notation: $s1$ denotes the total number of neurons per layer; $s2$ denotes the number of unstable neurons per layer; $s3$ denotes the cumulative number of unstable neurons starting from the input layer; $s4$ denotes the cumulative number of binary variables required to compute the interval for a single neuron, accumulated from the input layer up to the previous layers; $s5$ denotes the cumulative number of binary variables required to compute the intervals for all neurons in the current layer, accumulated from the input layer up to the previous layers, and it satisfies $s5 = s1 \times s4$. These metrics visualize the reduction in binary variables achieved by the SART encoding and support the subsequent analysis of runtime differences.

A key observation is consistent with Figure 4 in the main paper. Although the last two layers often have a smaller total number of binary variables $s5$, their solving time is typically longer. The reason is that these layers contain fewer unstable neurons $s2$ while the cumulative binary variables per neuron $s4$ are the largest, which increases the difficulty and time of each single-neuron subproblem. In contrast, the first layer has many neurons $s1$, which leads to a larger $s5$, but its $s4$ is small, so each subproblem is easier and the overall time remains short. This explains why in Figure 4 in the main paper the first layer shows a higher bar for total binary variables yet requires less time, whereas the last two layers show lower bars yet take longer.

D Discussion on Generality

We summarize the generality of the encoding and present three complementary axes: activations, norm bounded perturbations and regions, and integration with other solvers.

Table 7. Columns: **In**=input layer, **Lk**=layer k ($k = 0 \ldots 6$). Rows: **S1**=Total neurons per layer; **S2**=Unstable neurons per layer; **S3**=Cumulative unstable neurons; **S4**=Cumulative binary vars for one neuron (up to previous layers); **S5**=Cumulative binary vars for all neurons in this layer.

ID	Stat	In	L0	L1	L2	L3	L4	L5	L6
ID72 (Standard MILP)	S1	784	100	100	100	100	100	10	9
	S2	0	18	24	30	20	22	2	0
	S3	0	18	42	72	92	114	116	116
	S4	0	0	36	84	144	184	228	232
	S5	0	0	3600	8400	14400	18400	2280	2088
ID72 (Pure SART)	S1	784	100	100	100	100	100	10	9
	S2	0	18	24	30	20	22	2	0
	S3	0	18	42	72	92	114	116	116
	S4	0	0	18	60	114	164	206	230
	S5	0	0	1800	6000	11400	16400	2060	2070
ID18 (Standard MILP)	S1	784	100	100	100	100	100	10	9
	S2	0	19	30	18	13	8	0	0
	S3	0	19	49	67	80	88	88	88
	S4	0	0	38	98	134	160	176	176
	S5	0	0	3800	9800	13400	16000	1760	1584
ID18 (Pure SART)	S1	784	100	100	100	100	100	10	9
	S2	0	19	30	18	13	8	0	0
	S3	0	19	49	67	80	88	88	88
	S4	0	0	19	68	116	147	168	168
	S5	0	0	1900	6800	11600	14700	1680	1512

Table 8. Columns: **In**=input layer, **Lk**=layer k ($k = 0 \ldots 7$). Rows: **S1**=Total neurons per layer; **S2**=Unstable neurons per layer; **S3**=Cumulative unstable neurons; **S4**=Cumulative binary vars for one neuron (up to previous layers); **S5**=Cumulative binary vars for all neurons in this layer.

ID	Stat	In	L0	L1	L2	L3	L4	L5	L6	L7
ID50 (Standard MILP)	S1	784	100	100	100	100	100	100	10	9
	S2	0	31	25	12	5	1	3	0	0
	S3	0	31	56	68	73	74	77	77	77
	S4	0	0	62	112	136	146	148	154	154
	S5	0	0	6200	11200	13600	14600	14800	1540	1386
ID50 (LayerABS, Our SART)	S1	784	100	100	100	100	100	100	10	9
	S2	0	31	25	12	9	3	11	0	0
	S3	0	31	56	68	77	80	91	91	91
	S4	0	0	31	87	62	33	21	17	11
	S5	0	0	3100	8700	6200	3300	2100	170	99
ID89 (LayerABS, Std. MILP)	S1	784	100	100	100	100	100	100	10	9
	S2	0	33	31	42	37	33	30	1	0
	S3	0	33	64	106	143	176	206	207	207
	S4	0	0	66	128	146	158	140	126	62
	S5	0	0	6600	12800	14600	15800	14000	1260	558
ID89 (LayerABS, Our SART)	S1	784	100	100	100	100	100	100	10	9
	S2	0	33	31	42	35	33	29	1	0
	S3	0	33	64	106	141	174	203	204	204
	S4	0	0	33	97	104	119	103	95	59
	S5	0	0	3300	9700	10400	11900	10300	950	531

1. **Activation functions.** Although our development centers on ReLU, the encoding extends directly to any piecewise linear activation that admits an absolute value representation $f(x) = p\,x + q\,|x| + r$ with fixed coefficients. Leaky ReLU, PReLU, and RReLU all fit this template, so the constraints are instantiated by substituting the corresponding coefficients (for example, $p = \frac{1+\alpha}{2}$, $q = \frac{1-\alpha}{2}$, and $r = 0$). For the standard smooth Tanh, SART does not provide complete support if we require algebraic exactness, since mixed arithmetic involving transcendental functions is generally undecidable , as indicated by Richardson's theorem. In practice, the impact is limited because lightweight models often use Hard-Tanh as an efficient substitute, and Hard-Tanh can be exactly expressed as $0.5(|x + 1| - |x - 1|)$, making it amenable to exact SART verification. The mappings are summarized in Table 9.
2. **Norm bounded perturbations and regions.** Vector norms are used only to define the input perturbation set. The encoding of unstable ReLU neurons does not depend on the chosen norm. The method therefore applies to ℓ_∞, ℓ_1, and ℓ_2. More broadly, if the outer verifier can describe an input or output region, the encoding works with it. Common cases include polyhedral sets and norm balls. These choices do not change the core encoding.
3. **Integration with other solvers.** The encoding is a low level, theory driven scheme that operates at the constraint level. It can be instantiated within complete and incomplete verification pipelines, including mixed integer linear programming workflows, branch and bound workflows, and relaxation based workflows. Representative systems include Marabou, MIPVerify, β-CROWN BaB, and PRIMA, where the encoding can replace or strengthen the treatment of unstable ReLUs.

Table 9. Absolute value representations for common piecewise linear activations.

Activation	Piecewise Definition	Absolute Value Form	Typical Use Cases
ReLU	$f(x) = \max(0, x)$	$f(x) = \dfrac{x + \lvert x\rvert}{2}$	Default choice for hidden layers in MLPs and CNNs
Leaky ReLU	$f(x) = \max(\alpha x, x)$ (α usually 0.01)	$f(x) = \dfrac{1+\alpha}{2}x + \dfrac{1-\alpha}{2}\lvert x\rvert$	Direct replacement for ReLU, especially in GANs
PReLU	$f(x) = \max(\alpha x, x)$ (α is learnable)	$f(x) = \dfrac{1+\alpha}{2}x + \dfrac{1-\alpha}{2}\lvert x\rvert$	Large scale networks (for example, ResNet)
RReLU	$f(x) = \max(\alpha x, x)$ (α randomly sampled)	$f(x) = \dfrac{1+\alpha}{2}x + \dfrac{1-\alpha}{2}\lvert x\rvert$	Scenarios requiring regularization
Hard-Tanh	$f(x) = \mathrm{clamp}(x, -1, 1)$	$f(x) = \frac{\lvert x+1\rvert - \lvert x-1\rvert}{2}$	lightweight models